

网络安全——

网络攻击——拒绝服务攻击

北京邮电大学

武 斌

什么是DoS?



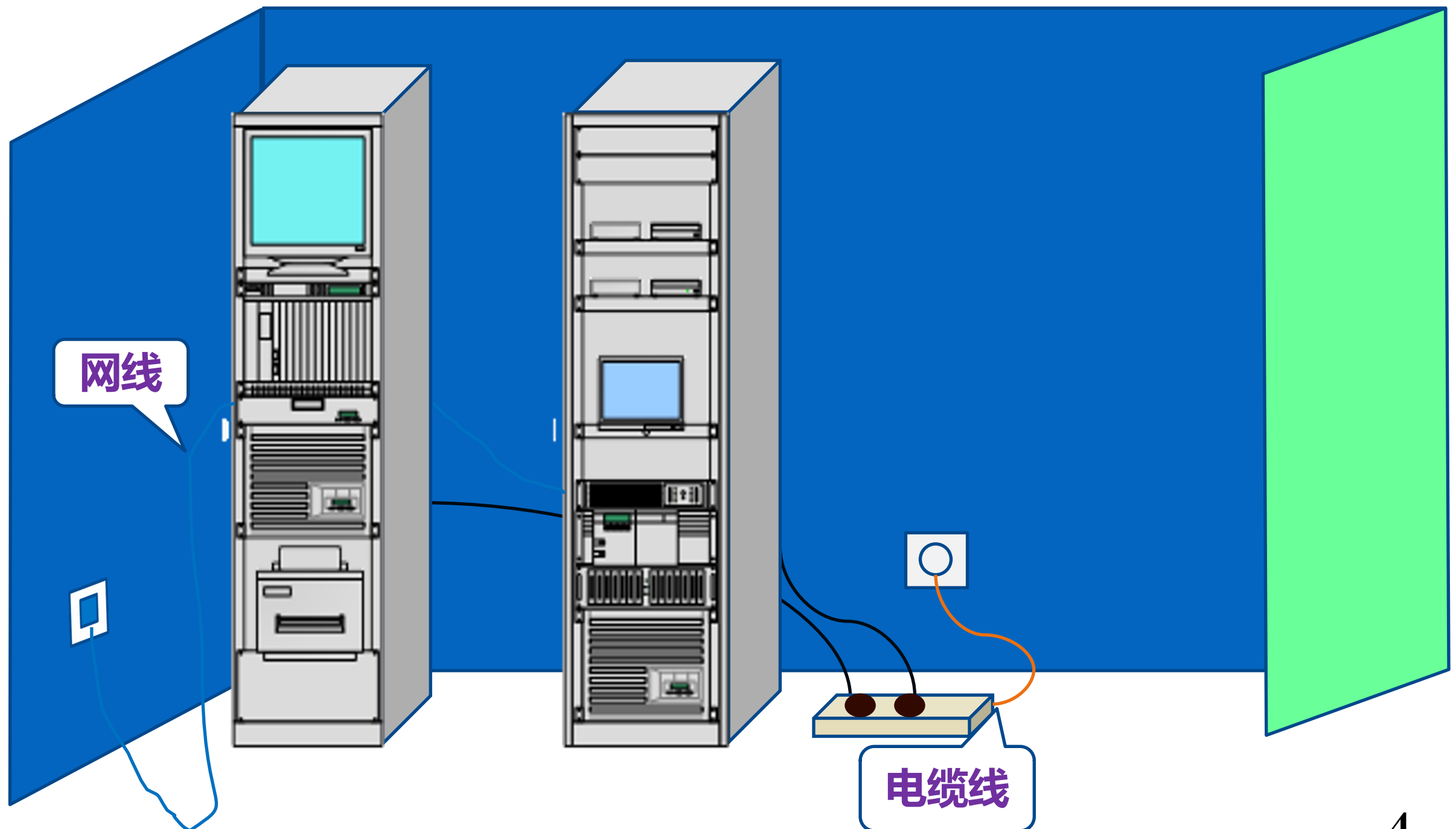
Denial of Service, 拒绝服务

什么是DoS?

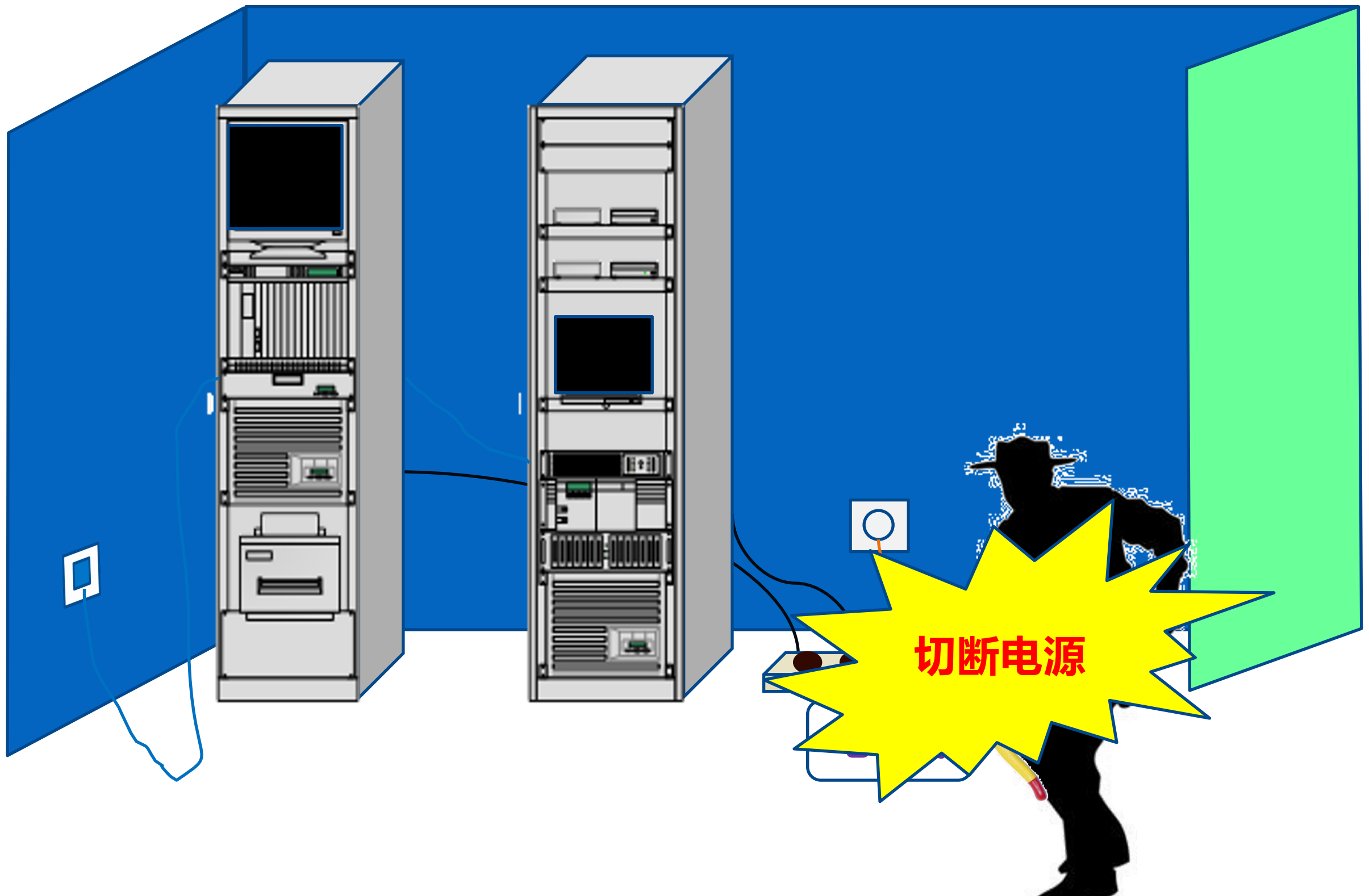
生活中形形色色的DoS

- “呼死你” “聊死你”
- “水军”
- 影院“锁场”
- 验证码轰炸

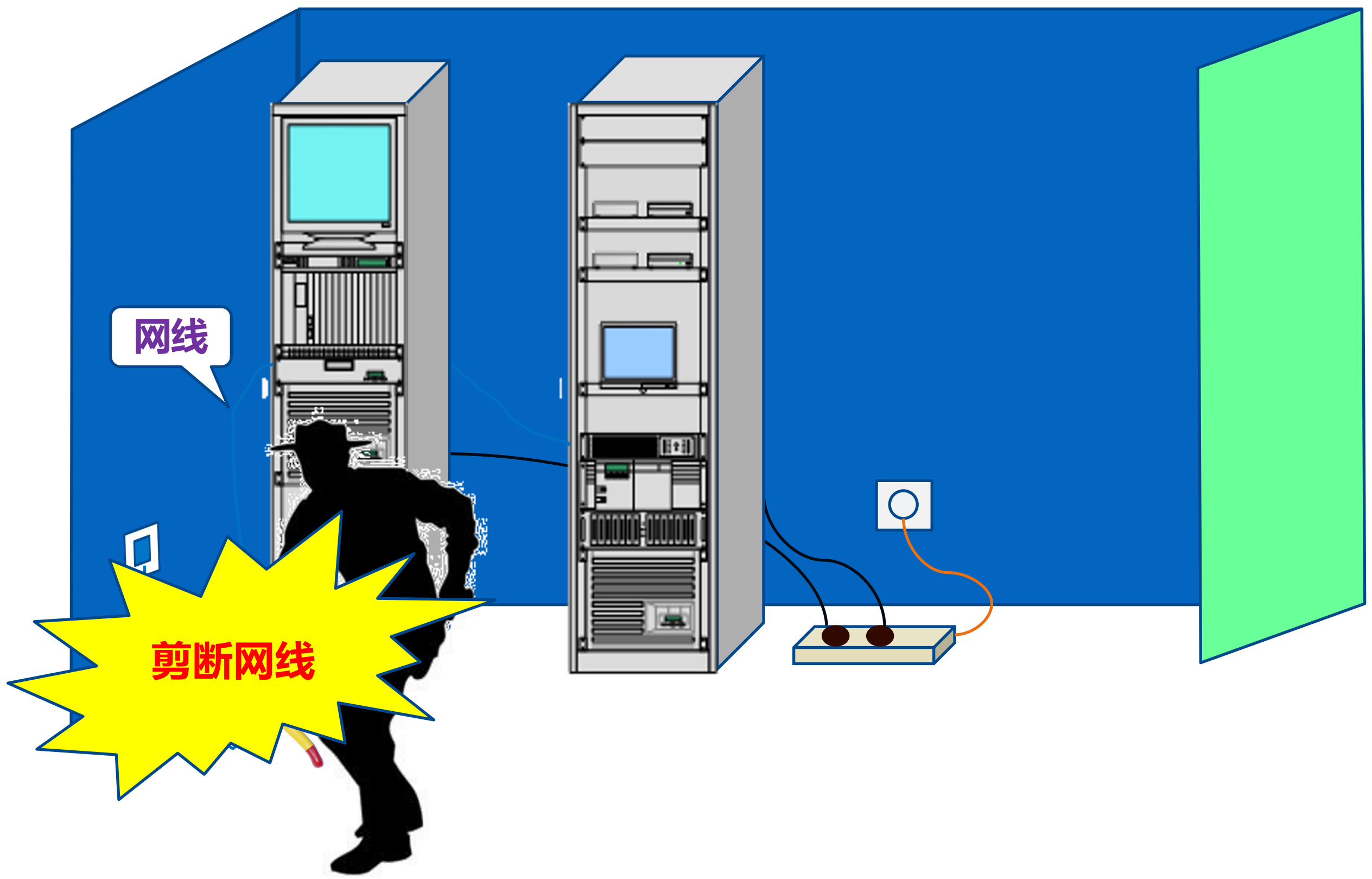
DoS/DDoS的概念



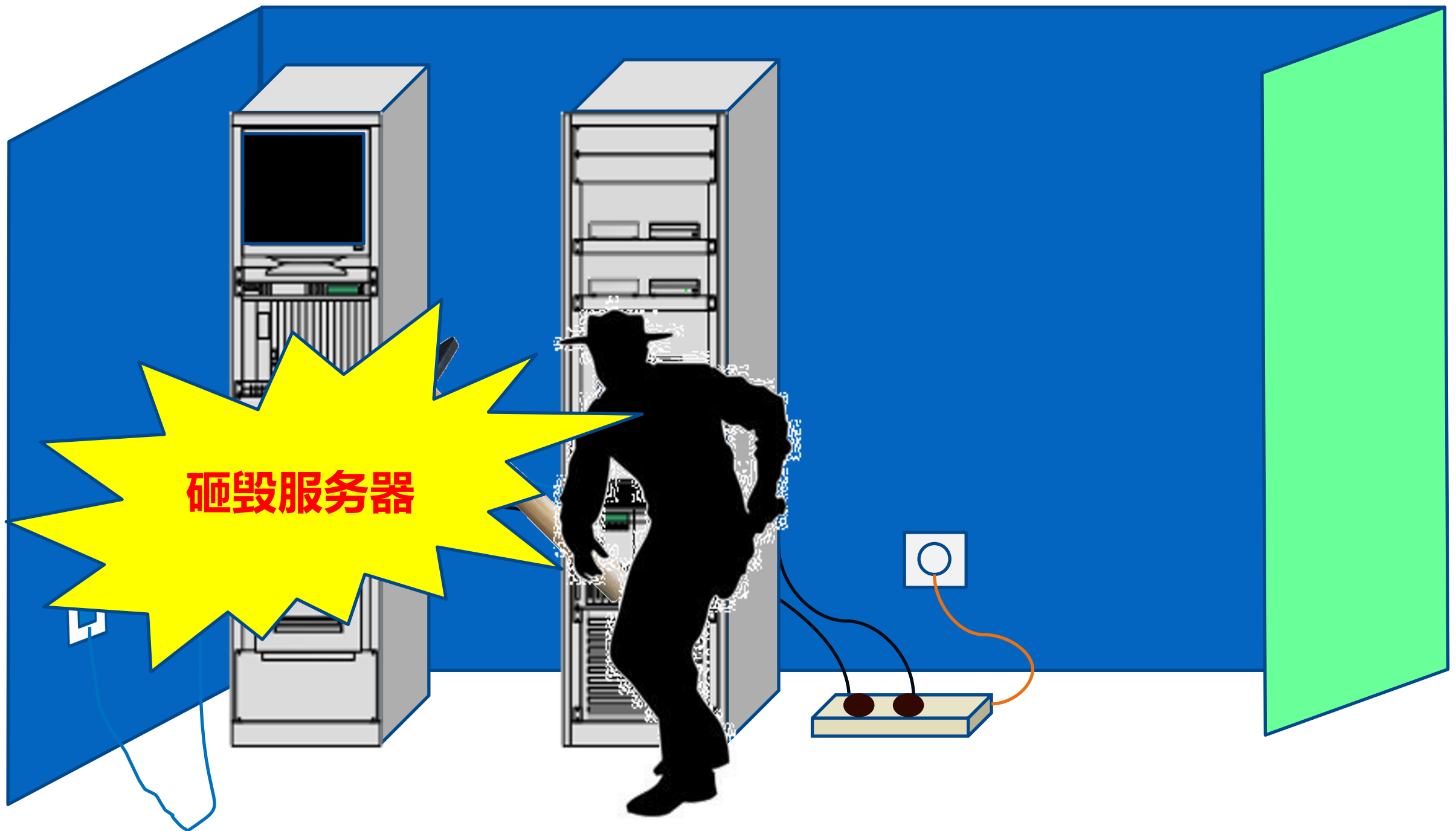
DoS/DDoS的概念



DoS/DDoS的概念

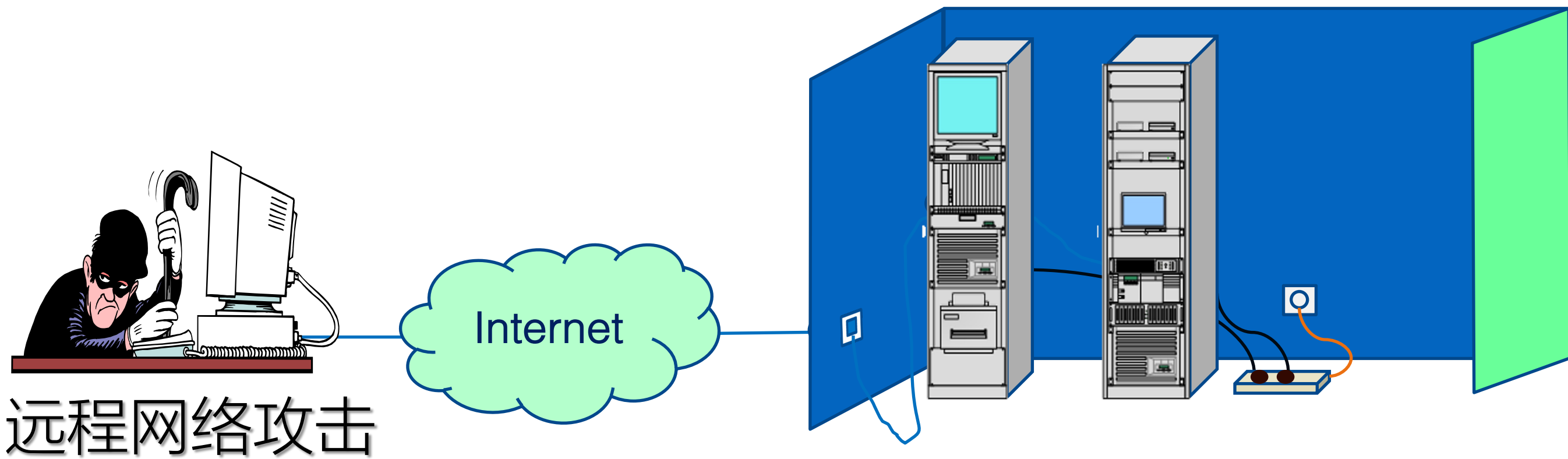


DoS/DDoS的概念



DoS/DDoS的概念

DoS ?



拒绝服务攻击——

大家熟悉的DoS？

典型的拒绝服务攻击

- Ping of Death
- Teardrop
- Land
- Syn Flood
- Smurf
- HTTP Flood
- CC

典型DoS—Ping of Death

- Ping is a computer network administration software utility used to **test the reachability** of a host on an Internet Protocol (IP) network.
- It measures the round-trip time for messages sent from the originating host to a destination computer that are echoed back to the source.
- **Ping operates by sending Internet Control Message Protocol (ICMP/ICMP6) Echo Request packets to the target host and waiting for an ICMP Echo Reply.** The program reports errors, packet loss, and a statistical summary of the results, typically including the minimum, maximum, the mean round-trip times, and standard deviation of the mean.

典型DoS—Ping of Death

The ***echo request*** ("ping") is an ICMP/ICMP6 message.

0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2	3	3		
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
Type = 8(IPv4, ICMP) 128(IPv6,ICMP6)								Code = 0								Header Checksum															
Identifier																Sequence Number															
Payload																															

The *echo reply* is an ICMP message generated in response to an echo request.

0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	2	3	3				
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1		
Type = 0(IPv4,ICMP) 129(IPv6,ICMP6)									Code = 0									Header Checksum															
Identifier																Sequence Number																	
Payload																																	

典型DoS—Ping of Death

- A ping of death is a type of attack on a computer system that involves sending a malformed or otherwise malicious ping to a computer.
- 早期的操作系统处理ICMP分组时，只开辟64KB缓冲区用来存放收到的数据包。
- 攻击者故意在ICMP Echo数据包(Ping包)之后附加非常多的冗余信息，使数据包的尺寸超过65535个字节的上限。
- 接收方对这种数据包进行处理时就会出现内存分配错误，导致TCP/IP堆栈溢出，从而引起系统崩溃，挂起或重启。

```
C:\Users\bupt>ping -l 66000 192.168.0.201  
选项 -l 的值有错误，有效范围从 0 到 65500。
```

典型DoS——Teardrop

- A teardrop attack involves sending mangled IP fragments with overlapping, oversized payloads to the target machine.
- This can crash various operating systems because of a bug in their TCP/IP fragmentation re-assembly code.^[54] Windows 3.1x, Windows 95 and Windows NT operating systems, as well as versions of Linux prior to versions 2.0.32 and 2.1.63 are vulnerable to this attack.
- One of the fields in an IP header is the “fragment offset” field, indicating the starting position, or offset, of the data contained in a fragmented packet relative to the data in the original packet. If the sum of the offset and size of one fragmented packet differs from that of the next fragmented packet, the packets overlap.

典型DoS——Teardrop

Fragmentation and reassembly

Fragment	Total bytes	Header bytes	Data bytes	"More fragments" flag	Fragment offset (8-byte blocks)
1	2500	20	2480	1	0
2	2040	20	2020	0	310

an MTU of 1,500 bytes , For example, consider a Transport layer segment with size of 4,500 bytes.

Fragment	Total bytes	Header bytes	Data bytes	"More fragments" flag	Fragment offset (8-byte blocks)
1	1500	20	1480	1	0
2	1020	20	1000	1	185
3	1500	20	1480	1	310
4	560	20	540	0	495

We can use the last offset and last data size to calculate the total data size: $495 \times 8 + 540 = 3960 + 540 = 4500$.

https://en.wikipedia.org/wiki/IPv4#Fragmentation_and_reassembly

典型DoS——Teardrop

- MTU (maximum transfer unit, 最大传送单位) 限制传输数据的包大小, 大数据包需要分段。Teardrop攻击就是利用这种分割重组间的漏洞而产生的攻击方式。
- Teardrop指的是向目标机器发送损坏的IP包, 诸如重叠的包或过大的包载荷, 该攻击通过TCP/IP协议栈中分片重组代码中的bug来瘫痪各种不同的操作系统。
- Teardrop攻击使接收数据方重组数据包时, 出现数据包长度超大(如负值), 导致溢出。(假设数据包中第二片IP包的偏移量小于第一片结束的位移, 而且算上第二片IP包的Data, 也未超过第一片的尾部, 这就是重叠现象。)

典型DoS——Teardrop

第一片: Fragment offset=0; 数据包长度 (length)=ip.total-length-ip.headerlength=36。

IP 首部(20 字节)	UDP 首部(8 字节)	数据 1(28 字节)
--------------	--------------	-------------

第二片: Fragmentoffset=32; 数据包长度 (length)=ip.total-length-ip.headerlength=3。

IP 首部(20 字节)	数据 2
--------------	------

若两片重组,则第二片必然嵌于第一片内,

	1	8 9	32	35 36
IP 首部(20 字节)	UDP 首部(8 字节)	数据 1	数据 2	

这时第一片的 end=36, offset=0; 第二片的 end=32+3=35, 而正常情况下第二片的段偏移 offset=36, 这样造成 fp->len=end-offset=35-36=-1。此时调用 memcpy ((ptr + fp->offset), fp->ptr, fp->len) 时, 由于 fp->len 为负数, 会引起堆栈溢出。

CVE-2021-24086 IPv6拒绝服务漏洞

- Windows IPv6协议栈存在一处拒绝服务漏洞，此漏洞的根本原因是IPv6的嵌套分片机制中，当尝试**递归重组嵌套的分片时**会计算内部有效载荷中包含的所有扩展标头，当重组扩展头约为0xffff字节的数据包时，在IPv6 ReassembleDatagram中发生的NULL指针取消引用，发生崩溃。

影响版本

- 远程攻击者可通过向目标系统发送特制数据包来利用此漏洞，成功利用此漏洞可导致目标系统拒绝服务（蓝屏）。
- 漏洞影响系统版本

```
Windows 10 Version 20H2 for x64-based Systems
Windows 10 Version 20H2 for 32-bit Systems
Windows 10 Version 20H2 for ARM64-based Systems
Windows 10 Version 20H2 (Server Core Installation)
Windows 10 Version 2004 for 32-bit Systems
Windows 10 Version 2004 for ARM64-based Systems
Windows 10 Version 2004 for x64-based Systems
Windows 10 Version 2004 (Server Core installation)
Windows 7 for 32-bit Systems Service Pack 1
Windows 7 for x64-based Systems Service Pack 1
Windows 8.1 for 32-bit systems
Windows 8.1 for x64-based systems
Windows RT 8.1
```

```
Windows Server 2019
Windows Server 2019 (Server Core installation)
Windows Server 2016
Windows Server 2016 (Server Core installation)
Windows Server 2012
Windows Server 2012 (Server Core installation)
Windows Server 2012 R2
Windows Server 2012 R2 (Server Core installation)
Windows Server 2008 for 32-bit Systems Service Pack 2
```

复现实验

- **漏洞复现**

- 1.下载POC

- <https://github.com/0vercl0k/CVE-2021-24086>

- 2.防火墙开启，防护正常开启。

复现实验

- 漏洞复现

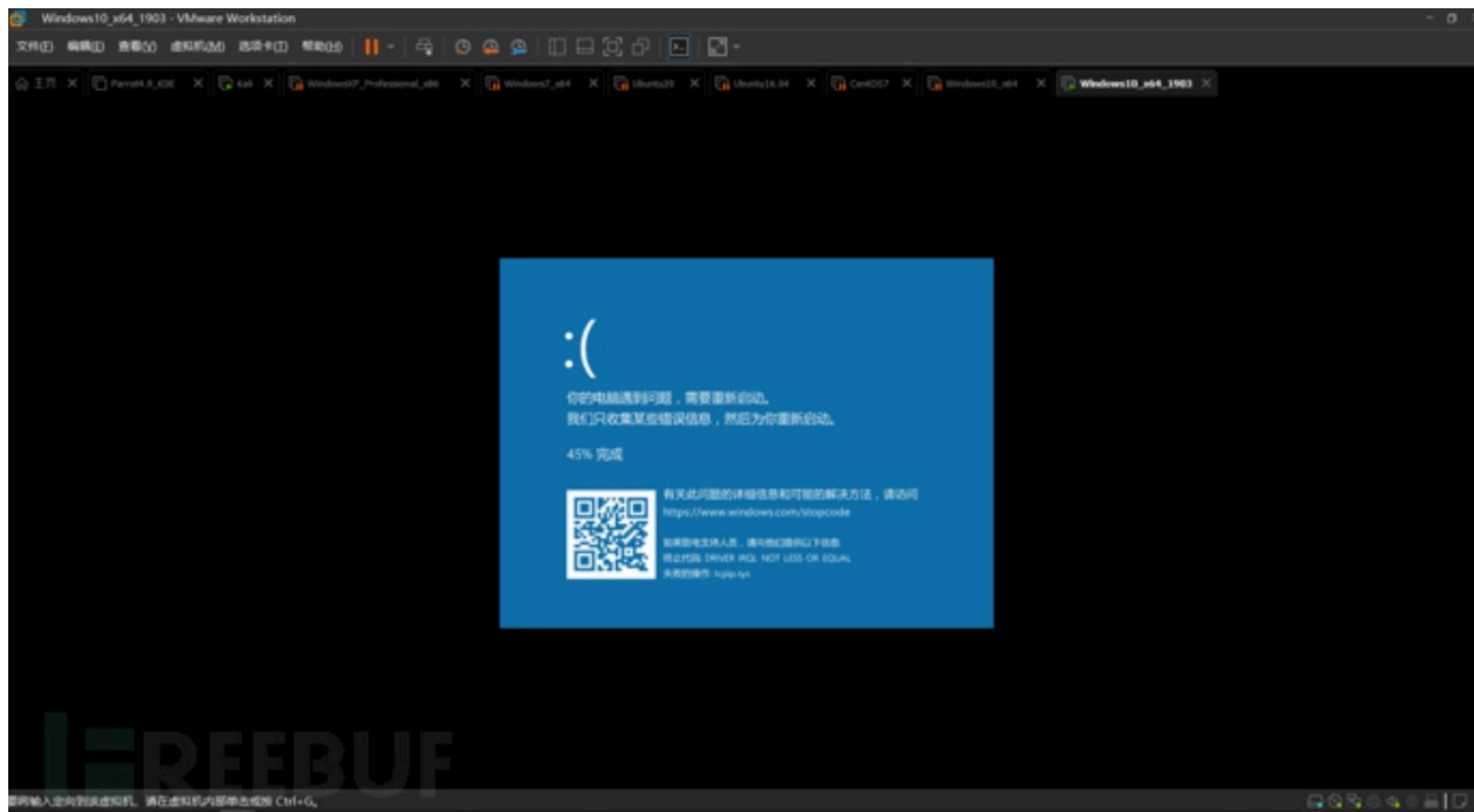
- 3.运行POC :

```
root@kali:~/Desktop/漏洞复现/CVE-2021-24086# python3 cve-2021-24086.py
66 fragments, total size 0xffff8
.....
Sent 66 packets.
.
Sent 1 packets.
root@kali:~/Desktop/漏洞复现/CVE-2021-24086# python3 cve-2021-24086.py --target
fe80::f13b:b0d2:f5f8:f1f0%14
66 fragments, total size 0xffff8
.....
Sent 66 packets.
.
Sent 1 packets.
root@kali:~/Desktop/漏洞复现/CVE-2021-24086#
```

复现实验

- 漏洞复现

- 4. 受害机器蓝屏：



缓解措施

微软官方措施

- 1. Set global reassembly limit to 0

The following command disables packet reassembly. Any out-of-order packets are dropped. Valid scenarios should not exceed more than 50 out-of-order fragments. We recommend testing prior to updating production systems.

```
Netsh int ipv6 set global reassemblylimit=0
```

- 2. Configure an Edge device, such as a firewall or load balancer, to disallow IPv6 fragmentation. Host based firewalls do not provide sufficient protection.

典型DoS——Land攻击

- Land攻击：利用特殊的TCP封包传送至目标主机，使其因无法判别而当机或被迫重新启动。
- 攻击原理是：用一个特别构造的SYN包，**它的源地址和目标地址都被设置成某一个服务器地址**。此举将导致接受服务器向它自己的地址发送SYN—ACK消息，结果这个地址又发回ACK消息并创建一个空连接。被攻击的服务器每接收一个这样的连接都将保留，直到超时。

典型DoS—SYN洪水

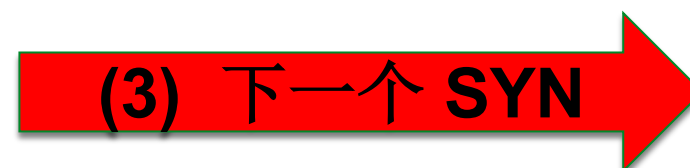
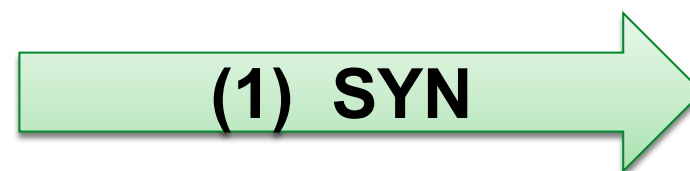
- SYN洪水

- 原理：

- 每个机器都需要为半开连接分配一定的资源
 - 这种半开连接的数量是有限制
 - 攻击方利用TCP连接三次握手过程，打开大量的半开TCP连接
 - 目标机器不能进一步接受TCP连接。机器就不再接受进来的连接请求。



攻击机



服务器

典型DoS—SYN洪水

- SYN洪水

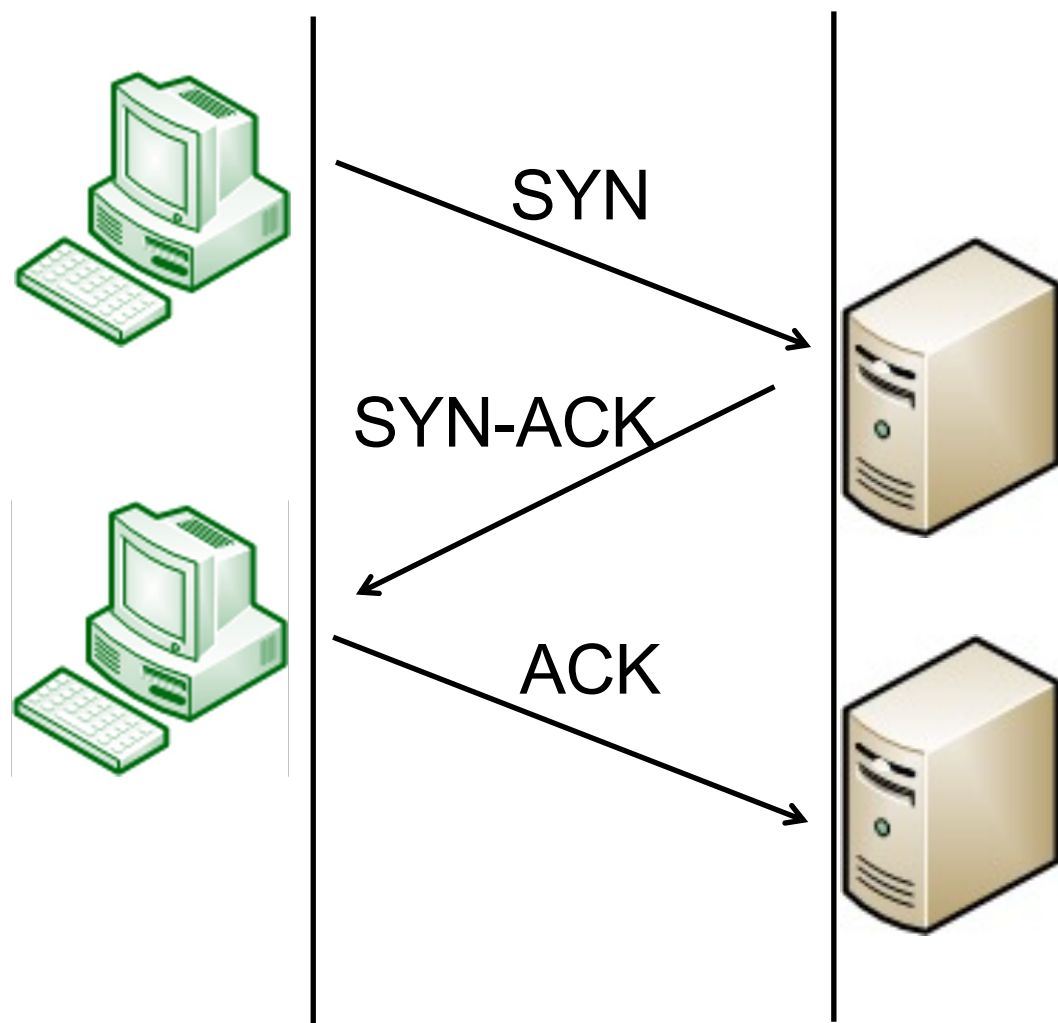
- 受影响的系统：大多数操作系统

- 攻击细节

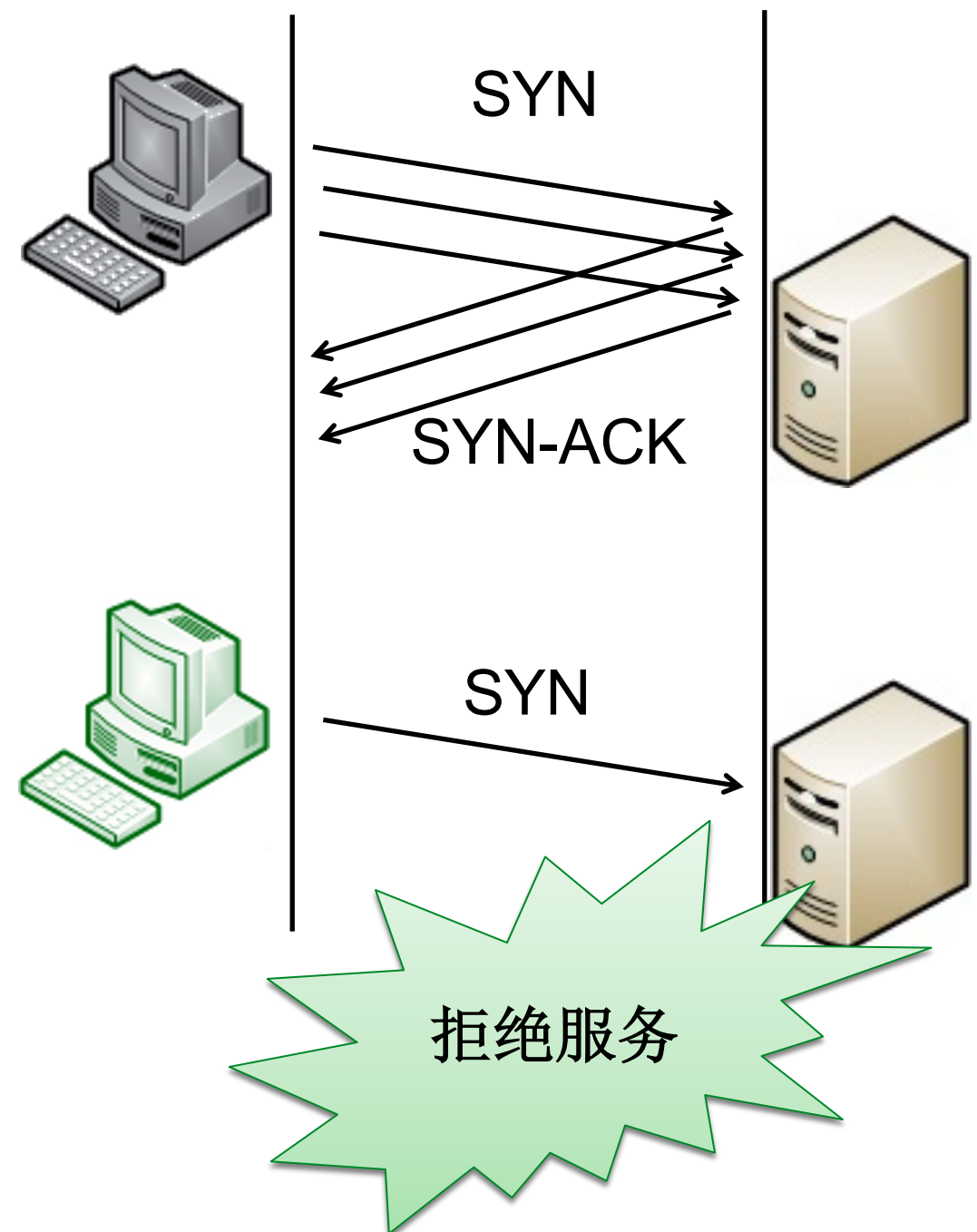
- 连接请求是正常的，但是，源IP地址往往是伪造的，并且是一台不可达的机器的IP地址，否则，被伪造地址的机器会重置这些半开连接
- 一般，半开连接超时之后，会自动被清除，所以，攻击者的系统发出SYN包的速度要比目标机器清除半开连接的速度要快
- 任何连接到Internet上并提供基于TCP的网络服务，都有可能成为攻击的目标
- 这样的攻击很难跟踪，因为源地地址往往不可信，而且不在线

典型DoS—SYN洪水

正常的SYN连接:



SYN/ACK Flood Attack:



典型DoS—Smurf

- Smurf

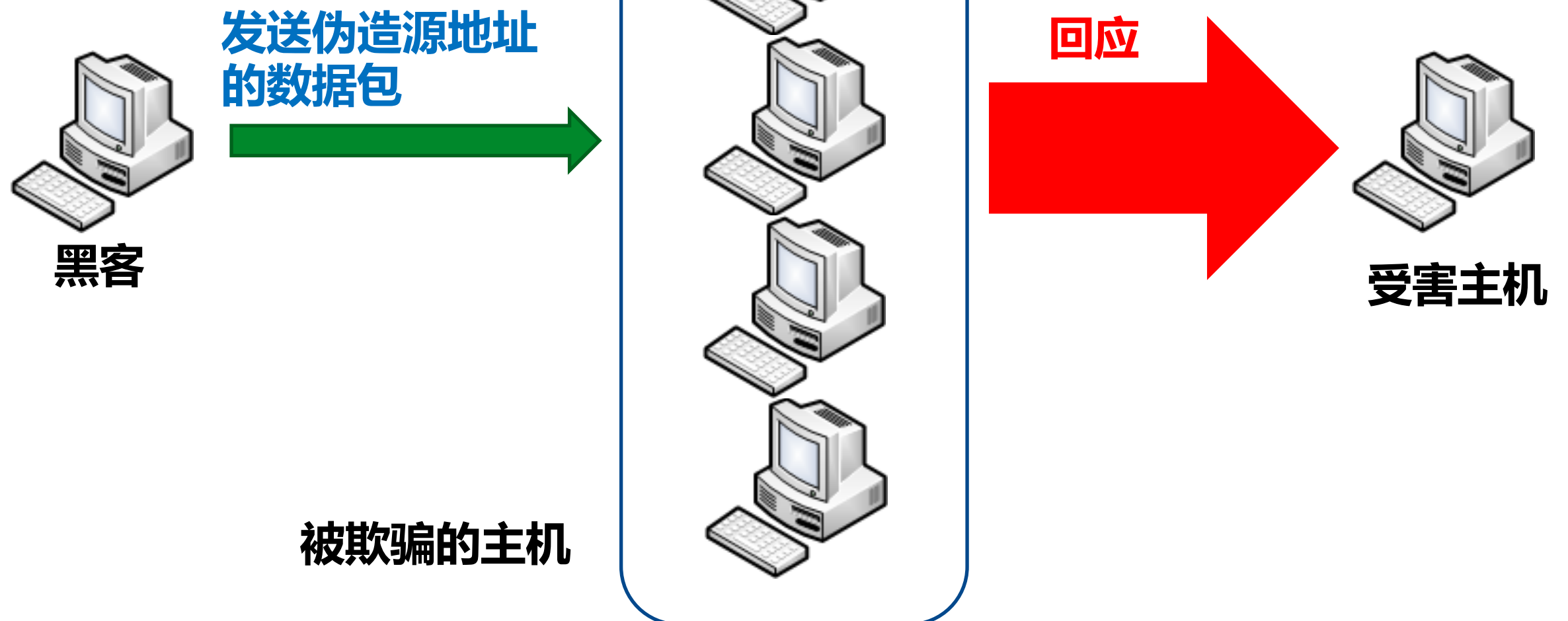
- 原理：

- 攻击者向一个广播地址发送ICMP Echo请求，并且**用受害者的IP地址作为源地址**
- 广播地址网络上的每台机器响应这些Echo请求，同时向受害者主机发送ICMP Echo-Reply应答
- 受害者主机会被这些大量的应答包淹没

- 受影响的系统：大多数操作系统和路由器

典型DoS—Smurf

- Smurf



典型DoS—HTTP泛洪

- HTTP泛洪
- 这类攻击会占用大量的HTTP进程，从而耗费大量的系统资源。最终，会导致系统因不堪重负而崩溃掉。
- 以最典型的HTTP GET FLOOD攻击为例。
 - HTTP GET FLOOD是针对应用服务器上的某个文件，对其进行快速的反复的重复读取操作，从而造成服务器的资源减少直至崩溃。
 - HTTP GET FLOOD针对的不仅仅是WEB服务器，还有数据库服务器。大量的HTTP请求产生了大量的数据库查询，可以在几秒之内使数据库停止响应，系统负载升高，最终导致服务器宕机。

传统DoS的变化

CC攻击：Challenge Collapsar

- 原理：CC攻击的原理就是攻击者控制某些主机不停地发大量数据包给对方服务器造成服务器资源耗尽，一直到宕机崩溃。
- 特点
 - 真实度高
 - 检测困难
 - 防御困难

小讨论：如何检测防护CC攻击？

传统DoS的变化

慢速攻击

- HTTP Post慢速DoS攻击：2012年的OWASP大会上，由Wong Onn Chee 和 Tom Brennan共同演示。
- 基本原理：对HTTP服务器，先建立了一个连接，指定一个比较大的content-length，然后以非常低的速度发包，比如1-10s发一个字节，然后维持住这个连接不断开。
- 如果客户端持续建立这样的连接，那么服务器上可用的连接将一点一点被占满，从而导致拒绝服务。

慢速HTTP拒绝服务攻击

- Slow headers

Web应用在处理HTTP请求之前，都要先接收完所有的HTTP头部，因为HTTP头部中包含了一些Web应用可能用到的重要的信息。攻击者利用这点，发起一个HTTP请求，一直不停的发送HTTP头部，消耗服务器的连接和内存资源。

- Kali上安装测试工具SlowHttpTest

```
sudo apt-get install slowhttpptest
```

- `slowhttpptest -c 1000 -H -g -o my_header_stats -i 10 -r 200 -t GET -u https://x.x.x.x -x 24 -p 3`

慢速HTTP拒绝服务攻击

- Slow headers

9	0.002048	10.67.3.220	10.67.3.215	TCP	74	55673 > xfer [SYN, Seq=0 win=29200 Len=0 MSS=
10	0.002058	10.67.3.215	10.67.3.220	TCP	78	xfer > 55673 [SYN, ACK] Seq=0 Ack=1 win=16384
11	0.003012	10.67.3.220	10.67.3.215	TCP	66	55673 > xfer [ACK] Seq=1 Ack=1 win=29312 Len=
12	0.003027	10.67.3.220	10.67.3.215	TCP	294	55673 > xfer [PSH, ACK] Seq=1 Ack=1 win=29312
2337	0.157268	10.67.3.215	10.67.3.220	TCP	66	xfer > 55673 [ACK] Seq=1 Ack=229 win=65307 Len=
2418	0.187146	10.67.3.220	10.67.3.215	TCP	74	55673 > xfer [PSH, ACK] Seq=229 Ack=1 win=29312
3049	0.360263	10.67.3.215	10.67.3.220	TCP	66	xfer > 55673 [ACK] Seq=1 Ack=237 win=65299 Len=
3219	30.045284	10.67.3.220	10.67.3.215	TCP	74	55673 > xfer [PSH, ACK] Seq=237 Ack=1 win=29312
4628	30.220488	10.67.3.215	10.67.3.220	TCP	66	xfer > 55673 [ACK] Seq=1 Ack=245 win=65291 Len=
4785	60.046920	10.67.3.220	10.67.3.215	TCP	74	55673 > xfer [PSH, ACK] Seq=245 Ack=1 win=29312
6079	60.188855	10.67.3.215	10.67.3.220	TCP	66	xfer > 55673 [ACK] Seq=1 Ack=253 win=65283 Len=
6369	90.048112	10.67.3.220	10.67.3.215	TCP	74	55673 > xfer [PSH, ACK] Seq=253 Ack=1 win=29312
7778	90.268052	10.67.3.215	10.67.3.220	TCP	66	xfer > 55673 [ACK] Seq=1 Ack=261 win=65275 Len=

```
Stream Content
GET / HTTP/1.1
Host: 10.67.3.215
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; windows NT 5.1; Trident/4.0; .NET CLR
1.1.4322; .NET CLR 2.0.50313; .NET CLR 3.0.4506.2152; .NET CLR 3.5.30729; MSoffice 12)
Content-Length: 42
X-a: b
```

慢速HTTP拒绝服务攻击

- Slow body

攻击者发送一个HTTP POST请求，该请求的Content-Length头部值很大，使得Web服务器或代理认为客户端要发送很大的数据。服务器会保持连接准备接收数据，但攻击客户端每次只发送很少量的数据，使该连接一直保持存活，消耗服务器的连接和内存资源。

- `slowhttptest -c 3000 -B -g -o my_body_stats -i 110 -r 200 -s 8192 -t FAKEVERB -u https://x.x.x.x -x 10 -p 3`

慢速HTTP拒绝服务攻击

- Slow body

	Time	Source	Destination	Protocol	Length	Info
24	174.765851	10.67.3.220	10.67.3.215	TCP	74	57517 > xfer [SYN] Seq
25	174.765873	10.67.3.215	10.67.3.220	TCP	78	xfer > 57517 [SYN, ACK]
27	174.766844	10.67.3.220	10.67.3.215	TCP	66	57517 > xfer [ACK] Seq
30	174.786000	10.67.3.220	10.67.3.215	TCP	401	57517 > xfer [PSH, ACK]
69	174.955818	10.67.3.215	10.67.3.220	TCP	66	xfer > 57517 [ACK] Seq
292	184.665462	10.67.3.220	10.67.3.215	TCP	76	57517 > xfer [PSH, ACK]
350	184.799597	10.67.3.215	10.67.3.220	TCP	66	xfer > 57517 [ACK] Seq
412	194.665660	10.67.3.220	10.67.3.215	TCP	107	57517 > xfer [PSH, ACK]
470	194.862163	10.67.3.215	10.67.3.220	TCP	66	xfer > 57517 [ACK] Seq
532	204.665869	10.67.3.220	10.67.3.215	TCP	115	57517 > xfer [PSH, ACK]
590	204.815263	10.67.3.215	10.67.3.220	TCP	66	xfer > 57517 [ACK] Seq
652	214.666034	10.67.3.220	10.67.3.215	TCP	119	57517 > xfer [PSH, ACK]
710	214.877693	10.67.3.215	10.67.3.220	TCP	66	xfer > 57517 [ACK] Seq

```
Stream Content
POST /py HTTP/1.1
Host: 10.67.3.215:82
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; windows NT 6.1; Trident/4.0; SLCC2)
Referer: http://code.google.com/p/slowhttpptest/
Content-Length: 4096
Connection: close
Content-Type: application/x-www-form-urlencoded
Accept: text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
```


慢速HTTP拒绝服务攻击

- Slow read

客户端与服务器建立连接并发送了一个HTTP请求，客户端发送完整的请求给服务器端，然后一直保持这个连接，**以很低的速度读取Response**，比如很长一段时间客户端不读取任何数据，通过发送Zero Window到服务器，让服务器误以为客户端很忙，直到连接快超时前才读取一个字节，以消耗服务器的连接和内存资源。

- `slowhttptest -c 8000 -X -r 200 -w 512 -y 1024 -n 5 -z 32 -k 3 -u https://x.x.x.x -p 3`

慢速HTTP拒绝服务攻击

- Slow read

228	0.725321	10.67.3.220	10.67.3.215	TCP	74	57705 > xfer [SYN] Seq=0 win=1152 Len=0 MSS=1460 SACK_PERM=1 TSval=
229	0.725338	10.67.3.215	10.67.3.220	TCP	78	xfer > 57705 [SYN, ACK] Seq=0 Ack=1 win=16384 Len=0 MSS=1460 WS=1
231	0.726318	10.67.3.220	10.67.3.215	TCP	66	57705 > xfer [ACK] Seq=1 Ack=1 win=1152 Len=0 TSval=1165968476 TSe
236	0.745526	10.67.3.220	10.67.3.215	TCP	255	57705 > xfer [PSH, ACK] Seq=1 Ack=1 win=1152 Len=189 TSval=1165968
238	0.749634	10.67.3.215	10.67.3.220	TCP	1218	[TCP window Full] xfer > 57705 [ACK] Seq=1 Ack=190 win=65536 Len=1
239	0.750883	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindow] 57705 > xfer [ACK] Seq=190 Ack=1153 win=0 Len=0
385	3.130670	10.67.3.215	10.67.3.220	TCP	67	[TCP ZeroWindowProbe] xfer > 57705 [ACK] Seq=1153 Ack=190 win=65536
391	3.131685	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 57705 > xfer [ACK] Seq=1
499	7.943211	10.67.3.215	10.67.3.220	TCP	67	[TCP ZeroWindowProbe] xfer > 57705 [ACK] Seq=1153 Ack=190 win=65536
505	7.945054	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 57705 > xfer [ACK] Seq=1
622	17.568187	10.67.3.215	10.67.3.220	TCP	67	[TCP ZeroWindowProbe] xfer > 57705 [ACK] Seq=1153 Ack=190 win=65536
628	17.569160	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 57705 > xfer [ACK] Seq=1
766	36.708882	10.67.3.215	10.67.3.220	TCP	67	[TCP ZeroWindowProbe] xfer > 57705 [ACK] Seq=1153 Ack=190 win=65536
777	36.709824	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 57705 > xfer [ACK] Seq=1
969	75.208822	10.67.3.215	10.67.3.220	TCP	67	[TCP ZeroWindowProbe] xfer > 57705 [ACK] Seq=1153 Ack=190 win=65536
984	78.306760	10.67.3.220	10.67.3.215	TCP	66	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 57705 > xfer [ACK] Seq=1

```
Stream Content
GET /py/ HTTP/1.1
Host: 10.67.3.215:82
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.7; rv:5.
Firefox/5.0.1
Referer: http://code.google.com/p/slowhttpptest/

HTTP/1.1 200 OK
Date: wed, 22 Apr 2015 16:30:50 GMT
Server: Apache/2.4.4 (win32) OpenSSL/0.9.8y PHP/5.4.16
X-Powered-By: PHP/5.4.16
Set-Cookie: PHPSESSID123=it9c2lk0ifouds8ic2kuau50n5; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check
Pragma: no-cache
```

拒绝服务攻击——

DoS检测

DoS检测

- 按检测模式分类的检测方法
 - 基于误用的DDoS检测
 - 基于异常的DDoS检测
 - 混合模式DDoS检测
- 按算法部署位置分类的检测方法
 - 源端检测
 - 中间网络检测
 - 目的端检测
 - SDN下的DoS检测

基于误用的DDoS检测

- 基于误用的DDoS攻击检测是指事先收集已有DDoS攻击的各种特征，然后将当前网络中数据包的特征和各种攻击特征相互比较，如果特征匹配则发现DDoS攻击。
- 基于误用方法依赖于**攻击特征的选取**，一般用于检测利用漏洞型的DDoS攻击。
- 基于误用的DDoS检测主要是利用了**特征匹配、模型推理、状态转换和专家系统**的方法。
 - 特征匹配主要是利用各种DDoS的特征进行检测。
 - 模型推理也是利用DDoS攻击的特征进行检测。
 - 状态转换将DDoS攻击看成被系统监测的一系列系统状态转换和相对应的条件，攻击事件与系统状态不要求一一对应。
 - 专家系统将专家关于DDoS攻击检测的知识转换成特征库中的特征与规则。检测系统一旦认为网络中出现了与专家系统中攻击发生的条件相匹配的现象，就判定发生了攻击。

基于异常的DDoS检测

- 基于异常的DDoS攻击检测是指通过监视系统审计记录上系统使用的异常情况，可以检测出违反安全的事件。目前，大多数的DDoS攻击检测都属于异常检测。
- 基于异常的DDoS检测取决于**检测模型的建立**，不同的模型对应着不同的检测方式，主要包括**统计检测**、**模式预测**、**人工智能检测**、**机器学习检测**四种方法。
 - 统计检测的方法是用统计的方法计算出网络正常工作时流量的阈值，然后与当前网络流量进行比较，如果当前网络流量超过了阈值则说明可能发生了DDoS攻击。
 - 模式预测就是通过分析攻击发生前必然发生的一些现象来判断是否发生了DDoS攻击。
 - 人工智能检测方法主要包括数据挖掘、人工神经网络和模糊理论等。
 - 使用机器学习的方法实现DDoS攻击的检测也是可行的。

混合模式DDoS检测

- 混合模式DDoS攻击检测是将误用DDoS攻击检测和异常DDoS攻击检测**两种方式混合使用**。
- 通常使用数据挖掘的方法, 由异常检测发现攻击, 从发现的攻击中摘录特征放入误用模式特征库中, 再利用误用检测的方法来检测DDoS攻击。但实际效果根据具体情况的不同也有差异。

按算法部署位置分类的检测方法

- 源端检测

- 源端DDoS攻击检测指的是将检测算法布置在发出攻击数据包的主机所处网络的边界路由器上。
- 将DDoS攻击检测系统部署在源端,可以使得攻击数据流在进入网络之前被阻止。

- 中间网络检测

- 中间网络DDoS攻击检测是指将攻击检测算法部署在整个网络上,包括路由器、交换机或其他网络设备。
- 在中间网络进行检测,通常是在核心路由器上部署分布式的DDoS防御检测系统。

- 目的端检测

- 目的端DDoS攻击检测是指将攻击检测算法部署在被攻击的主机和相关网络设备上。目前应用得最多的攻击检测都是在目的端(即受害端)进行的。

DDoS攻击检测方法的分析

表 1 DDoS 攻击检测技术比较

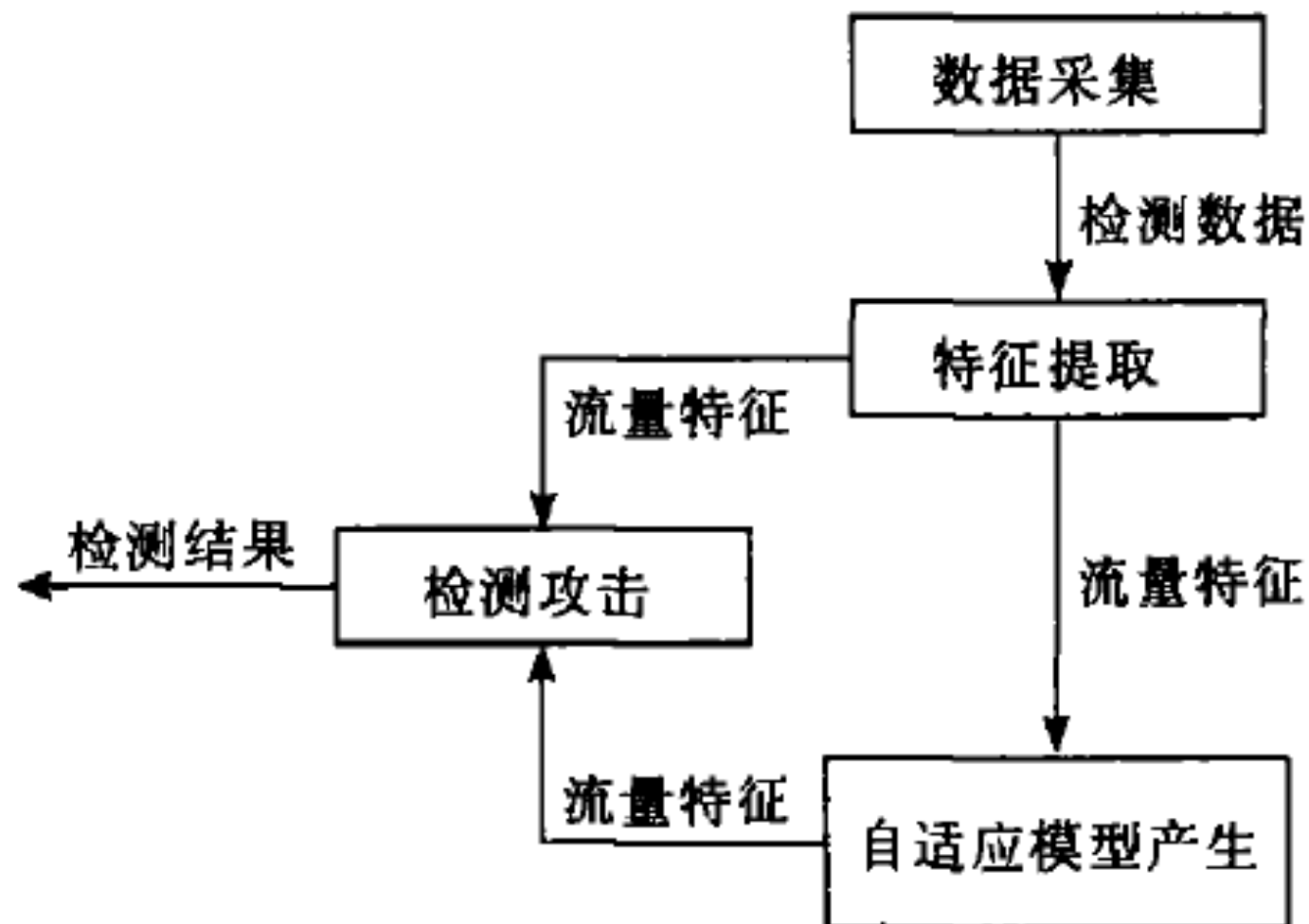
DDoS 检测技术	指 标						
	检测率	误报率	漏报率	收敛时间	算法复杂度	建模难易	维护开销
特征匹配	相当高	相当低	高	实时	低	较易	较大
模型推理	较高	较低	较高	较快	较高	难	较大
状态转换	较低	较低	较高	快	较低	较易	较小
专家系统	较低	较低	较高	较慢	较高	难	较大
统计	较高	低	较低	较快	较低	较易	较小
模式预测	较高	较低	低	较慢	高	较难	较小
系统调用	较低	相当低	高	快	低	较易	较小
数据挖掘	较高	较低	较低	较慢	较高	较易	较大
神经网络	较高	较低	较低	较快	较低	较易	较小
模糊理论	较高	较低	较低	快	低	较难	较小

表 2 DDoS 攻击检测算法部署位置比较

部署 位置	指 标						
	检测率	误报率	漏报率	部署难易	路由器 开销	正常报文 存活率	管理开销
源端	较低	较低	较高	难	较小	大	较小
中间网络	较高	较低	较低	难	较小	较大	大
目的端	高	较高	低	易	大	小	较小

基于数据挖掘的拒绝服务攻击检测技术

- 利用数据挖掘算法中的关联算法和聚类算法分步处理数据, 能够自动地、实时地、有效地检测DoS 攻击。



基于数据挖掘的拒绝服务攻击检测技术

- 连接记录格式

时间	源 IP 地址	目的 IP 地址	源端口	目的端口	协议类型	连接结束标志
----	---------	----------	-----	------	------	--------

图 3 连接记录的内容

- 检测模型产生

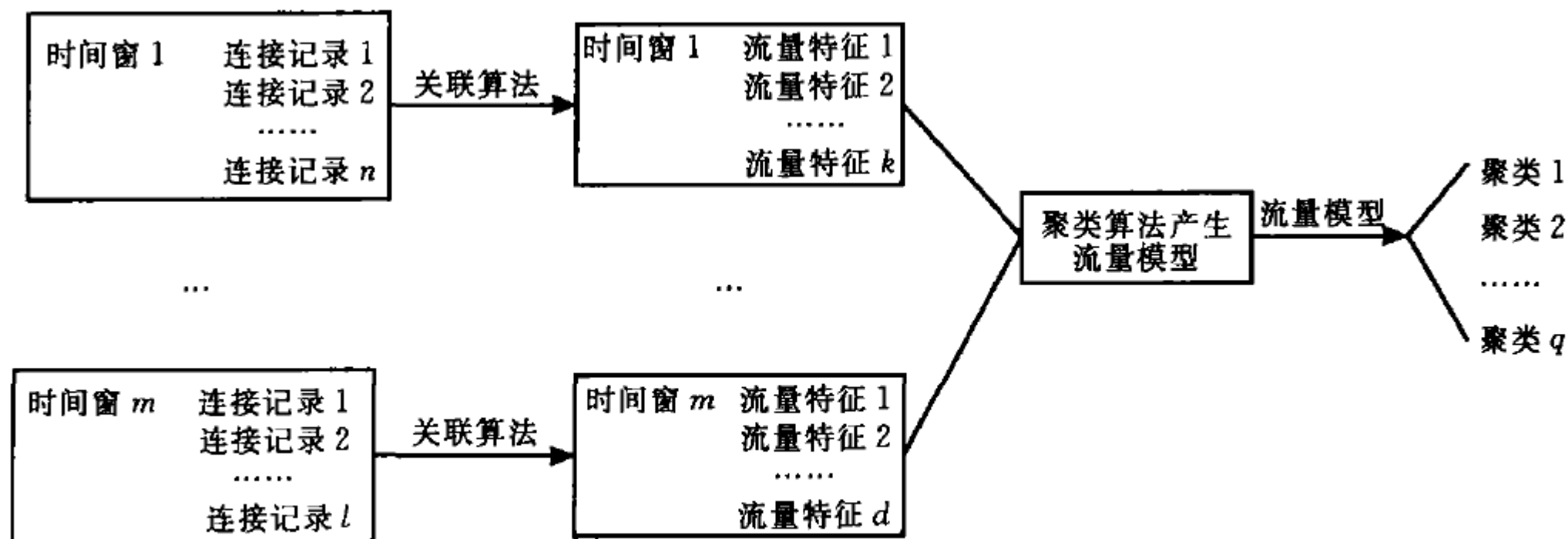


图 2 检测模型产生过程示意图

基于数据挖掘的拒绝服务攻击检测技术

- 流量特征

表 1 流量特征的内容

特征名称	特征含义
Service	服务类型(目的端口号)
dstIP	目的 IP 地址
Status	连接结束状态标志
count_conn	该时间窗内具有相同的 service、dstIP 和 status(即服务类型、目的 IP 和状态标志)的连接记录的个数
Count_total_conn	该时间窗内连接记录总个数

- 聚类模型

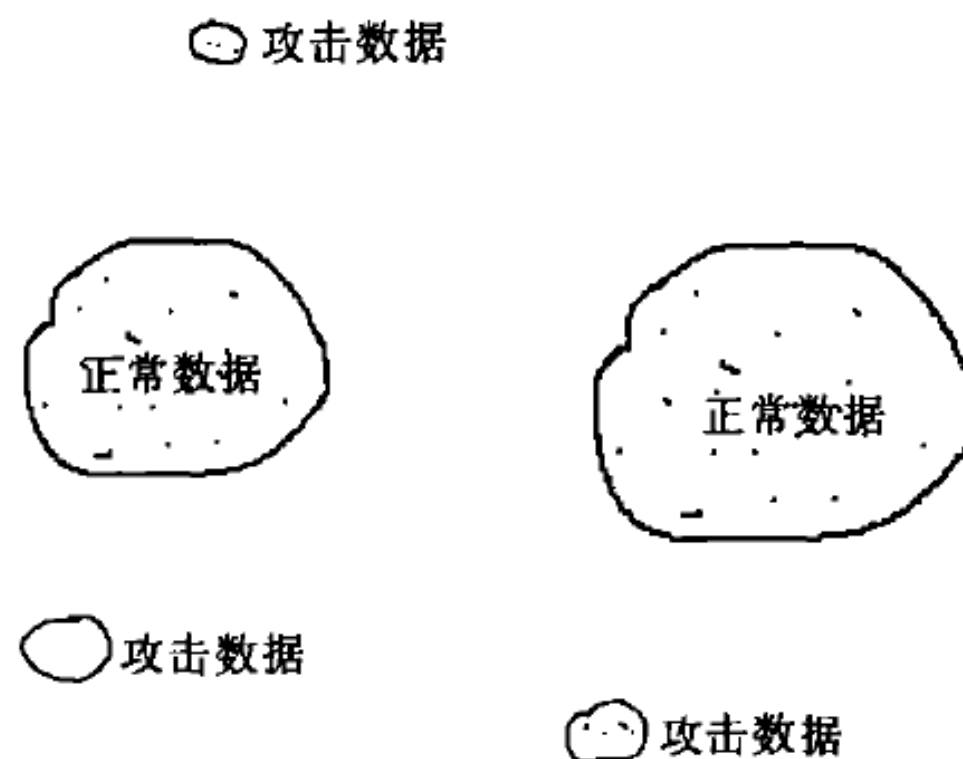


图 4 聚类模型示意图

低速率拒绝服务(low-rate denial of service)

- 当前研究的LDoS攻击：针对TCP 拥塞控制机制的攻击
不需要维持持续的高速攻击流，利用网络协议或应用服务协议的**自适应机制中**存在的安全漏洞，通过周期性地发送高速脉冲攻击数据包，达到降低受害端的服务性能的目的。
- 显著的特点：
 - 攻击目标是各种自适应机制，攻击引起的反应和调整是合法的
 - 攻击流量与许多真实的数据流特征类似，隐蔽性好
 - 攻击成本低，一个单一的攻击源就可以发动一次攻击，需要发送的数据远小于洪泛式DoS 攻击

低速率拒绝服务(low-rate denial of service)

- TCP拥塞控制机制
 - 慢启动(slow start)、拥塞避免(congestion avoidance)
 - 快速重传(fast retransmit)、快速恢复(fast recovery)、超时重传 (retransmission time out)
 - 发送端会根据当前的链路拥塞情况动态地调整发送报文的速率

低速率拒绝服务(low-rate denial of service)

- LDoS 攻击正是利用了TCP 拥塞控制算法中的 **RTO**(retransmission time out)和**AIMD**(additive increase multiplicative decrease)两种自适应机制，故意制造网络拥塞状况，使拥塞控制一直处于调整状态，达到拒绝服务效果。
- 尽管TCP 拥塞控制考虑到了避免崩溃的安全问题，即设置了RTO 的最小值和最大值，但由于这两种自适应机制调整的值大小差异明显，因此在低速率拒绝服务攻击发生时，发送端的发送速率会迅速变小，导致受害端的服务性能显著降低。

低速率拒绝服务(low-rate denial of service)

- 单源攻击

利用上述两种自适应机制,发动周期性的脉冲攻击， T 代表攻击周期， R 代表攻击速率， L 代表单个攻击脉冲持续时间。当攻击强度($R \cdot L$)足够大时，就可以造成足够的报文丢失，从而引发RTO 或AIMD 机制，造成持续频繁的自适应调整，从而达到攻击的效果。

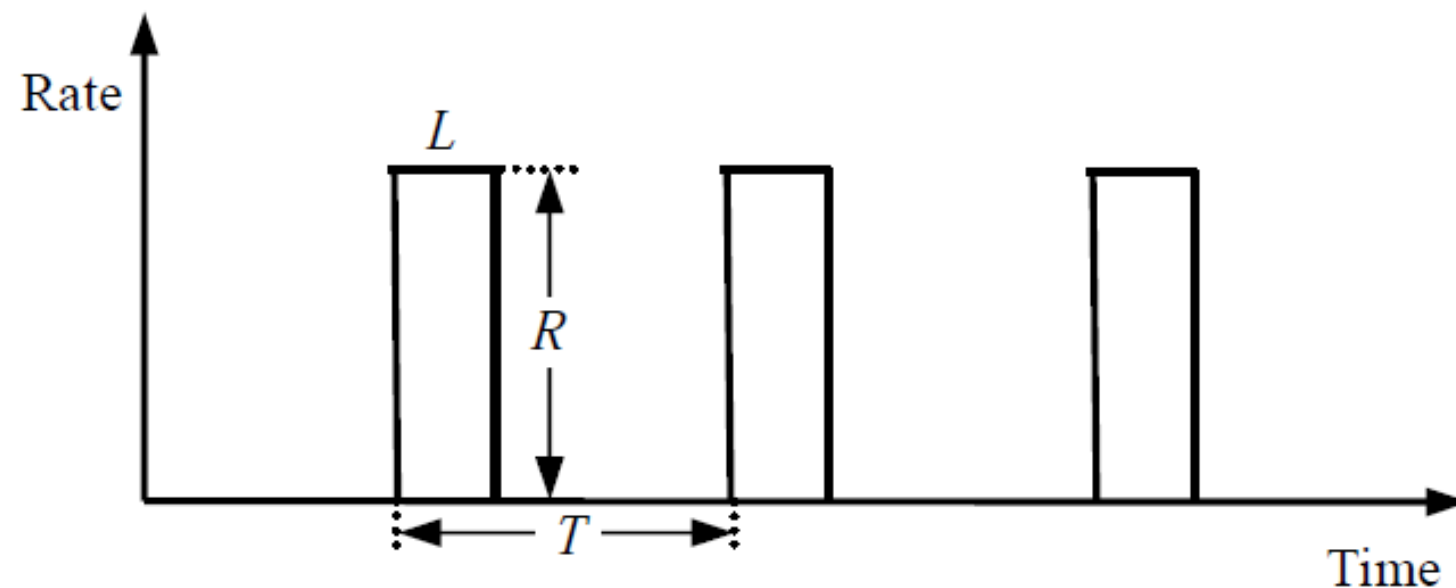


Fig.3 Pulse of LDoS attack

图 3 低速率拒绝服务攻击流示意图

低速率拒绝服务(low-rate denial of service)

- 多源攻击

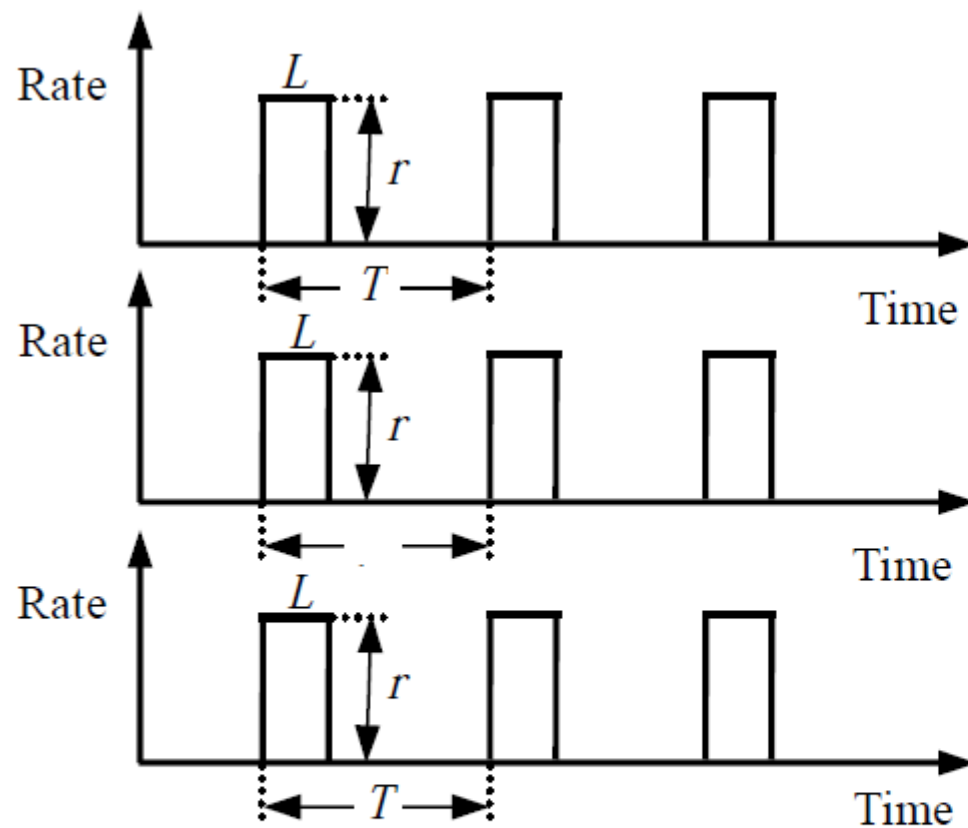


Fig.4 Synchronous DDoS attack

图 4 同步的 DDoS 攻击

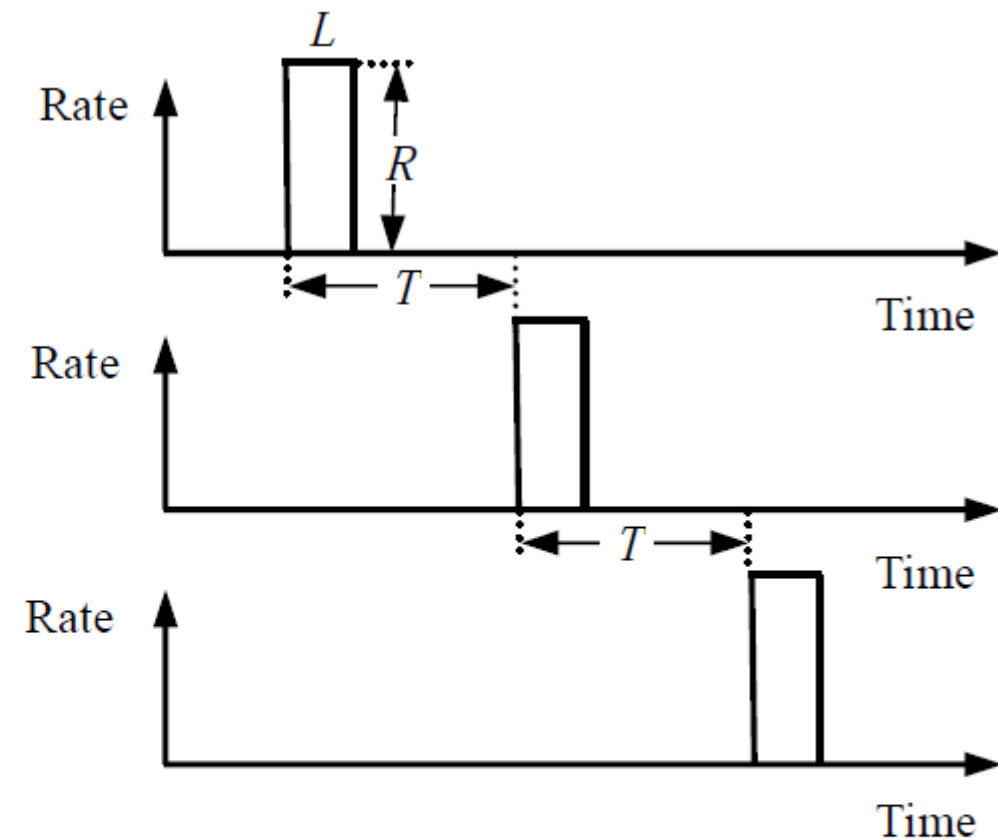


Fig.5 Asynchronous DDoS attack

图 5 异步的 DDoS 攻击

低速率拒绝服务(low-rate denial of service)

- 基于RTO机制的攻击

理论上讲，如果攻击者能够精确知道TCP 发送端每次发送报文时的RTO 值，就可以在其每次重传数据包的时候发动攻击脉冲，使得每次重传都超时，*cwnd* 始终保持为1，达到最大化攻击效果。

而实际上,根据TCP 协议RFC2988，RTO 值是一个动态值，需要根据往返时间RTT 更新RTO 值。

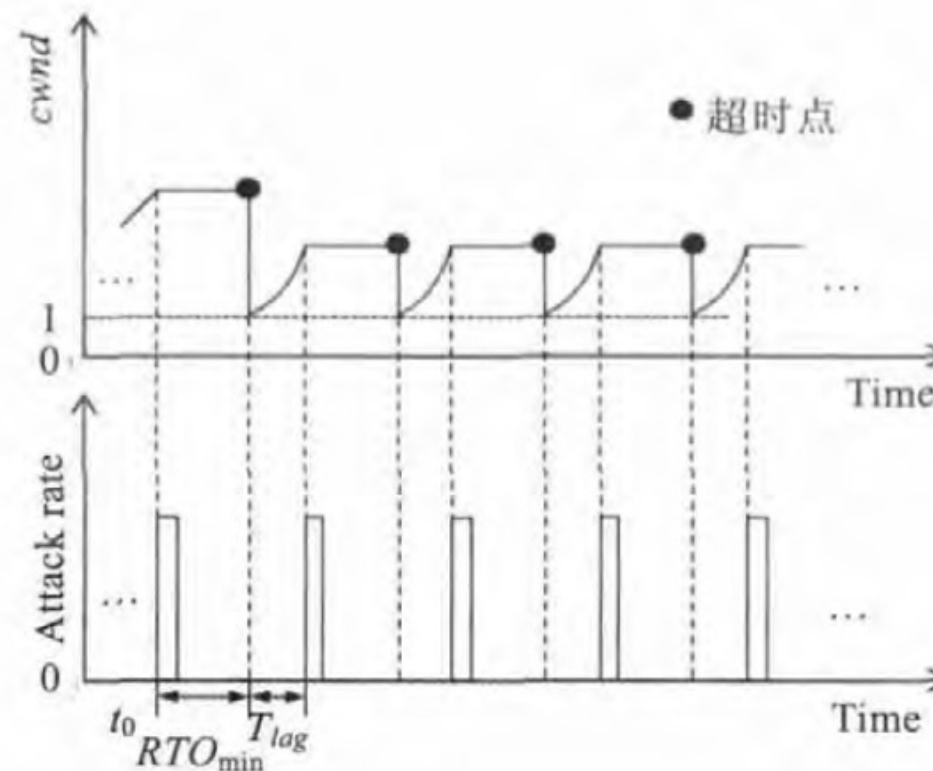


Fig.6 LDoS attack based on RTO mechanism

图 6 基于 RTO 重传机制的攻击

LDoS 攻击的检测

- 一般传统的 DoS 攻击检测方法：针对网络数据流和服务**器负荷激增等特征**进行检测
- 而LDoS 攻击表现出来许多不同的特征，比如：攻击数据流平均速率较低、服务器负荷不升反降等。传统的检测方法很难适用于检测LDoS 攻击
- 特征检测
- 异常检测

LDoS 攻击的检测

- **特征检测**
- LDoS攻击的脉冲强度、持续时间和攻击周期等特征也很明显，特别是结合其**周期性特征和短时高速脉冲特征**，能够很好检测到攻击。
- 最常见的防御策略就是**改进路由器的主动队列管理**(active queue management, AQM)
- 目标：一是丢弃符合设定攻击特征的数据流的数据包；二是重新进行带宽分配，尽量保护TCP 数据流，抑制LDoS 攻击流。

LDoS 攻击的检测

- 动态时间封装(Dynamic time warping,DTW)

五元组(T,L,R,S,N)描述攻击特征

- T 表示攻击的周期
- L 表示一个突发攻击流持续的时间
- R表示突发攻击流量的峰值速率
- S 表示计时开始到第一个突发攻击流量开始之间的时间差
- N 表示背景流量的等级

LDoS 攻击的检测

- **异常检测**

小波变换分析：离散小波变换

频谱分析：傅里叶变换、功率谱分析

统计分析：统计模型、马尔科夫模型、

信息度量分析：信息熵、广义熵、信息距离

小信号分析：小信号检测理论，特征估值矩阵

基于ACK 序号步长分析的攻击检测方法

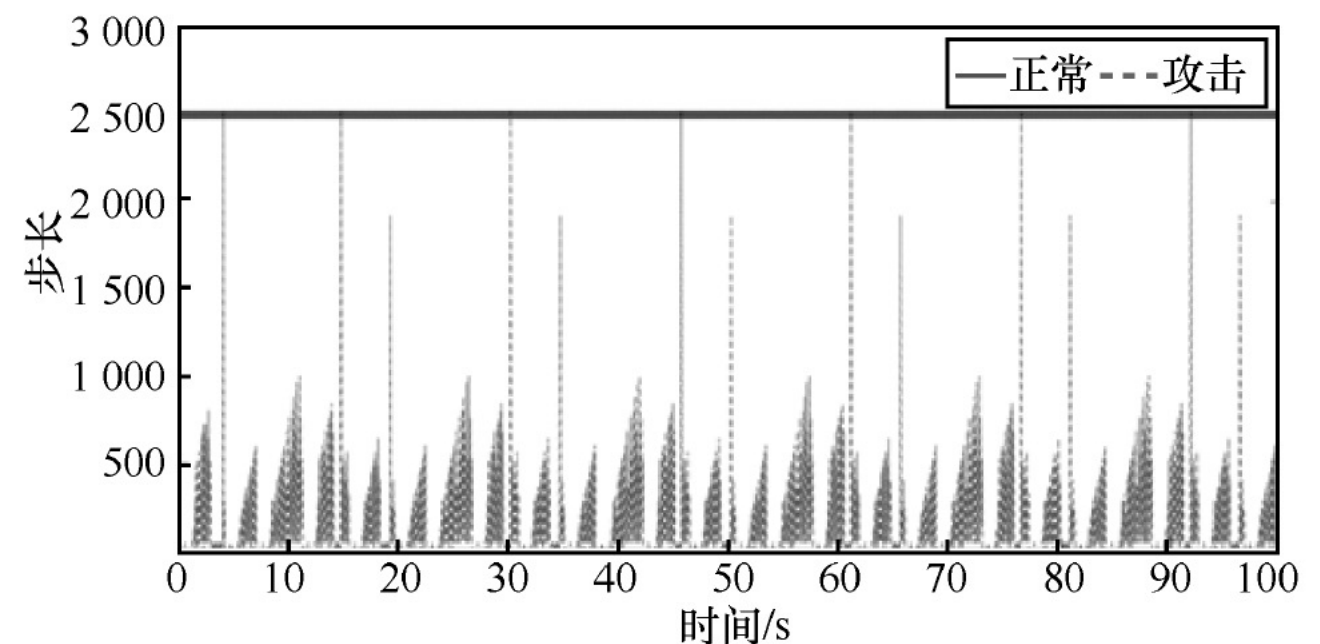
- 通过对正常网络和存在LDoS 攻击时ACK 数据分组的采样分析发现，**ACK 序号步长的波动程度**存在明显差异，且在遭受LDoS 攻击的瞬间，步长存在突变行为。针对这种波动差异，可以用熵值的方法对其进行度量并加以区分。

➤ ACK 序号步长特征分析

➤ 异常检测：基于**熵值**的突变

检测算法-排列熵算法

(PE, permutation entropy)



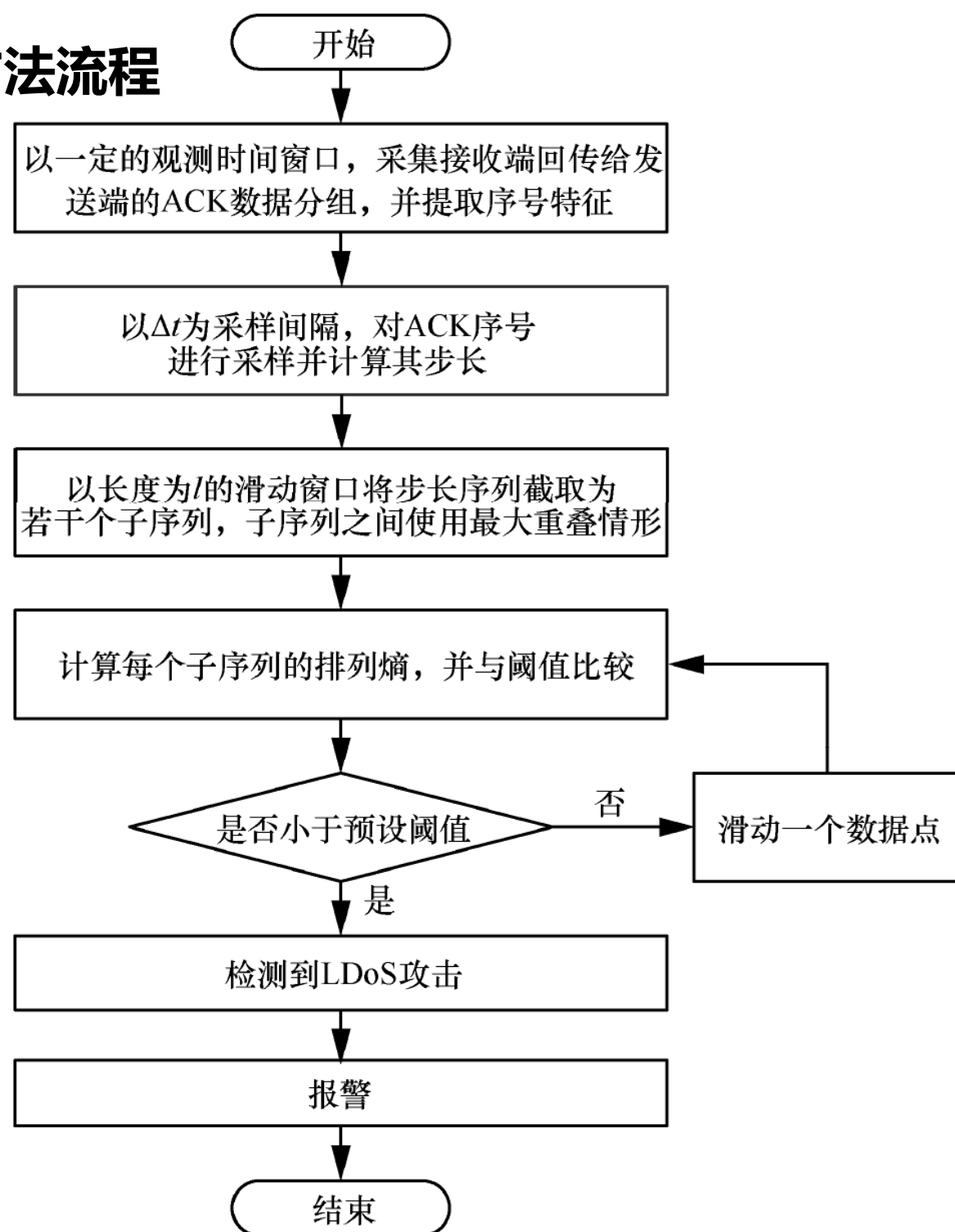
基于ACK 序号步长分析的攻击检测方法

- 先对一维时间序列进行相空间重构
- 再计算排列熵值

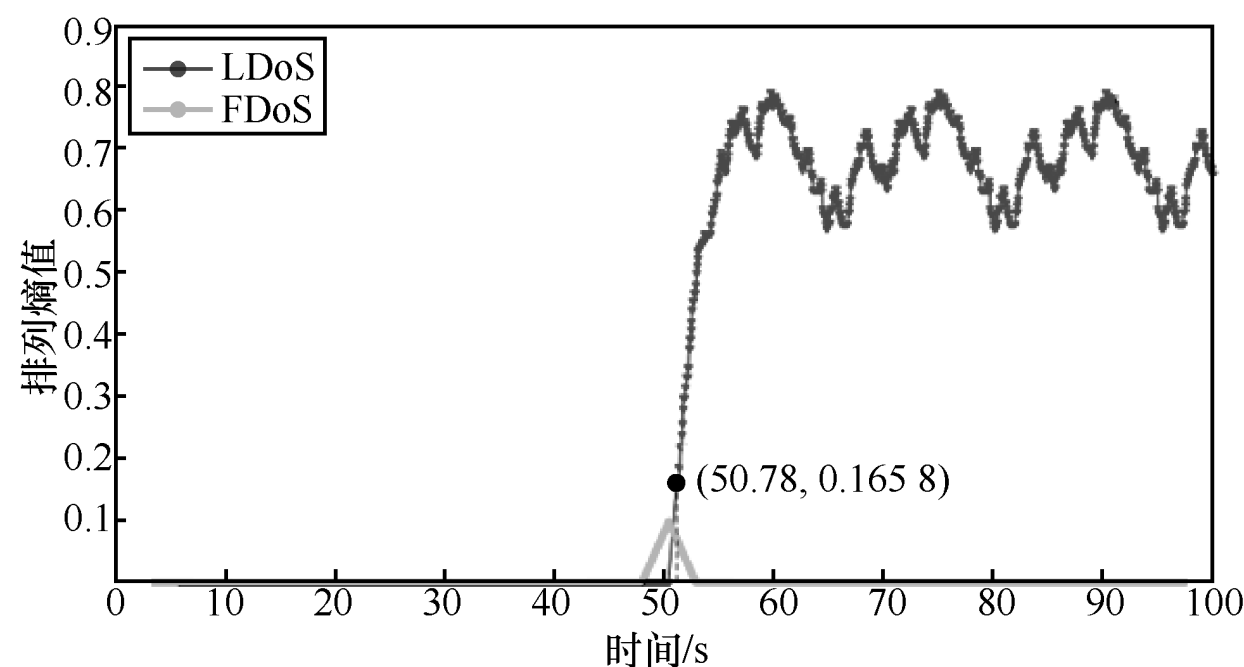
$$\begin{bmatrix} x(1) & x(1+\tau) & \cdots & x[1+(d-1)\tau] \\ x(2) & x(2+\tau) & \cdots & x[2+(d-1)\tau] \\ \vdots & \vdots & \vdots & \vdots \\ x(m) & x(m+\tau) & \cdots & x[m+(d-1)\tau] \\ \vdots & \vdots & \vdots & \vdots \\ x(k) & x(k+\tau) & \cdots & x[k+(d-1)\tau] \end{bmatrix}, m=1,2,3,\dots,k$$
$$H_P(d) = -\sum_{m=1}^k P_m \ln P_m$$

基于ACK 序号步长分析的攻击检测方法

方法流程



检测效果图



问题和讨论